

Deep Learning for Multivariate Time Series Forecasting

Group ID: 75

Nikolas Martin Diehl (2797049)

Ceylin Ekinici (3429785)

Kshitij Parashar (3440735)

Fabian Joel Zwinge (3758126)

August 30, 2026

Abstract

Multivariate time-series forecasting underlies many operational decisions. We study a benchmark in which an hourly operational load index is forecast across 96 series over a 336-hour horizon, with eleven domain covariates *known at inference* while the target is withheld. We compare gradient-boosted trees, a gated multilayer perceptron (MLP), and a Temporal Fusion Transformer (TFT). The TFT attains the best validation WAPE (14.48), substantially improving on the naive last-value (48.10) and seasonal-mean (34.41) baselines, but only narrowly beating the gated MLP (14.75) and the LightGBM-MLP ensemble (14.68). This suggests a feature-set ceiling in which different architectures extract similar information. On a cow tri-axial accelerometer dataset, the same TFT architecture beats the naive baseline by roughly 32% (WAPE 43.0 vs. 63.8); an ablation shows a modest benefit from the known behaviour covariate.

1 Introduction

Multivariate time-series forecasting is relevant in operational settings where decisions depend on accurate long-horizon forecasts. In this project, we forecast an hourly operational load index across 96 time series over a horizon of 336 hours. Eleven future covariates are known at inference time, while the future target remains hidden. The task therefore differs from history-only forecasting and can be viewed as known-covariate sequence-to-sequence forecasting.

We investigate this problem across several model classes. Gradient-boosted decision trees [1] serve as traditional machine-learning baselines, while a gated multilayer perceptron (MLP) represents a generic deep-learning approach with learned feature selection. The central model is the Temporal Fusion Transformer (TFT) [2], designed for multi-horizon forecasting with static, historical, and known future inputs. Comparing these approaches allows us to examine whether performance is driven mainly by model architecture or by the available feature set. We further investigate whether the findings transfer to a smaller cow accelerometer dataset [3].

Our research questions are:

- **RQ1.** Can a deep-learning model outperform strong traditional machine-learning baselines (e.g. gradient-boosted trees) on this known-future-covariate task?
- **RQ2. (core)** Does an architecture purpose-built for known future covariates with variable selection (TFT) beat a generic gated MLP - or does a feature-set ceiling hold across architecture classes, with diverse models converging to the same function on fixed features?
- **RQ3.** Do the findings transfer to a second forecasting dataset (the cow accelerometer data, cast into the same setup), and does augmentation help more in that low-data regime?

2 Related Work

Time-series forecasting includes classical methods such as ARIMA, ETS, and seasonal averages, as well as machine-learning and deep-learning approaches. Gradient-boosted trees, such as LightGBM, can model nonlinear relationships but typically depend on manually engineered temporal

features [1]. Recurrent models such as LSTMs learn sequential dependencies directly from historical observations [4]. More recent architectures include PatchTST [5], iTransformer [6], and pretrained foundation models such as Chronos [7]. Qiu et al. [8] provide a broader overview of deep multivariate forecasting from a channel-strategy perspective.

Model complexity does not necessarily lead to better forecasts. DLinear, for example, demonstrates that simple linear models can compete with transformer architectures on long-horizon tasks [9]. The TFT addresses multi-horizon forecasting with heterogeneous inputs by separately processing static, historical, and known future variables. It also uses adaptive variable-selection networks and supports probabilistic forecasts through quantile outputs [2]. Another relevant research direction is time-series data augmentation, which includes techniques such as window sampling, perturbation, masking, and frequency-domain transformations and is particularly relevant in settings with limited training data [10].

3 Method

3.1 Input Representation and Feature Engineering

Before defining the final model inputs, we performed an exploratory data analysis (EDA) of the target, covariates, temporal patterns, correlations, missing values, and differences between the individual time series. Based on these observations and subsequent experiments, the final input representation combines historical target information, known future covariates, static series information, and engineered temporal features.

The eleven provided domain covariates are available to both the encoder and the full forecast horizon. In addition, first-order differences of `queue_pressure_forecast`, `network_pressure_forecast`, and `shock_risk` are included to capture short-term changes. Temporal information is represented through cyclical encodings of hour of day, day of week, and day of year, together with categorical hour and weekday features. The historical target is used only by the encoder, while `series_id` identifies the individual time series.

Missing covariate values are imputed separately within each series using forward filling followed by backward filling. The target is normalized per series using a group-based normalizer, while the training loss is computed on the original target scale.

3.2 Model Selection and Baselines

Following the initial data analysis and feature engineering, we explored several forecasting approaches to identify suitable model families. We first established statistical and machine-learning baselines, including LightGBM [1], and then evaluated deep-learning approaches such as a gated MLP and LSTM encoder-decoder models [4], PatchTST [5], and Chronos [7].

Some simpler approaches provided strong reference models and were retained as baselines for our research questions. In contrast, architectures such as PatchTST and Chronos were less promising in our initial setup and were not pursued further. These experiments narrowed the model space and provided the reference points against which our final forecasting architecture is evaluated.

3.3 Model Architecture

Our final model is based on the TFT, which combines recurrent sequence processing, variable selection, gating, and attention to model both short- and long-term dependencies in multivariate time series. Figure 1 illustrates the overall architecture. It consists of five main building blocks:

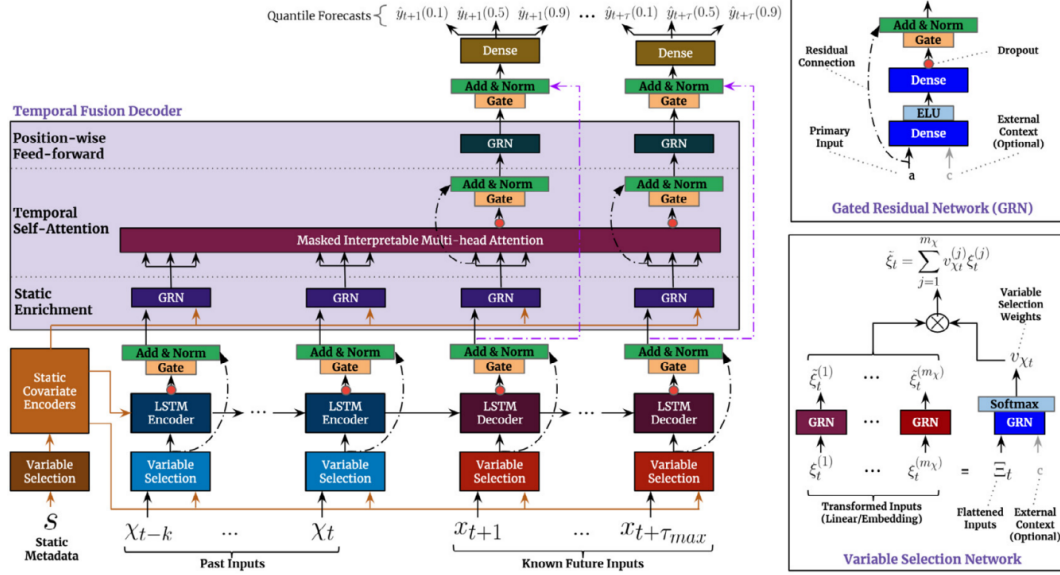


Figure 1: Overview of the TFT forecasting architecture. Figure from original paper [2].

1. **Gating Mechanisms.** Gating mechanisms control the information flow through the network and allow unnecessary transformations to be suppressed or skipped.
2. **Variable Selection Networks.** Variable selection networks determine which historical observations and known future covariates are most relevant at each time step.
3. **Static Covariate Encoding.** Static covariates are encoded into context vectors that can condition other parts of the network. These representations provide additional context for variable selection and temporal processing.
4. **Temporal Processing.** An LSTM-based sequence-to-sequence structure captures local and short-term temporal dependencies from historical observations and known future inputs. The resulting representations are subsequently processed by interpretable multi-head attention to capture longer-term relationships across time.
5. **Forecast Generation.** The model uses the P50 quantile, which represents the median and provides a single point forecast for each future time step. As the forecasting task requires one predicted value per timestamp, the model produces 336 P50 forecasts, which are subsequently evaluated using the different performance metrics considered in our experiments.

The final TFT uses a hidden size of 96, a hidden continuous size of 32, four attention heads, one LSTM layer, and a dropout rate of 0.15.

3.4 Training Setup

Training samples use an encoder history of up to 336 hours and a fixed prediction horizon of 336 hours. To increase training-window diversity, the encoder length is randomly varied between 168 and 336 hours, with windows sampled at a stride of 24 hours. Model selection uses three-fold rolling-origin validation while preserving temporal order. Each fold evaluates a 336-hour period,

while the preceding 336 hours are used exclusively for early stopping, ensuring that evaluation data do not influence model selection.

The model is optimized using MAE on the original target scale and AdamW with a learning rate of 10^{-3} and weight decay of 10^{-4} . We use a batch size of 32 with gradient accumulation over four batches, resulting in an effective batch size of 128. Training runs for up to 60 epochs with an early-stopping patience of 10, gradient clipping at 0.5, and dropout of 0.15. After model selection, the final models are trained on the full training period for 10 epochs, corresponding to the median best epoch from cross-validation. Five models with seeds 42, 7, 13, 99, and 1234 are trained and their predictions averaged to form a five-model ensemble. All models are trained using mixed precision on two NVIDIA Tesla T4 GPUs.

4 Experiments

4.1 Experimental Setup

The TFT model’s performance is evaluated using WAPE, MAE, MSE, RMSE, MAPE, and sMAPE. It is compared against last-value, seasonal-mean, LightGBM, and gated-MLP baselines.

4.2 Validation Results

Table 1 compares the forecasting approaches on the benchmark validation set.

Method	WAPE	MAE	RMSE	Notes
Naive last-value baseline	48.10	5.29	6.97	Minimum baseline
Seasonal mean baseline	34.41	3.79	5.21	Stronger reference
LightGBM	14.68	1.62	3.06	Machine-learning baseline
Gated MLP	14.75	1.62	3.06	Main reference model
TFT	14.48	1.59	3.08	Final architecture

Table 1: Validation results on the benchmark dataset. Lower is better for all listed metrics.

The TFT achieves a WAPE of 14.48 and an MAE of 1.59, substantially outperforming the naive last-value baseline (WAPE 48.10) and the seasonal-mean baseline (34.41). This corresponds to a WAPE reduction of approximately 70% and 58%, respectively. The gated MLP reaches a WAPE of 14.75, while the best LightGBM-MLP ensemble achieves 14.68.

Although the TFT therefore performs best in terms of WAPE and MAE, the differences between the learned models are small. Their RMSE values are also very similar, with the TFT at 3.08 compared with 3.06 for both alternatives. This suggests that much of the achievable performance is driven by the informative known future covariates, while the TFT’s variable selection and temporal modeling provide an additional but relatively modest improvement.

4.3 Analysis: Architecture versus Feature-Set Ceiling

The near-identical scores of the three learned models in Table 1 - TFT (14.48), the LightGBM-MLP ensemble (14.68), and the gated MLP (14.75), all within roughly 0.3 WAPE and with RMSE differing by 0.02 - act as a de-facto architecture ablation. Despite very different inductive biases, the models converge to almost the same predictive function once they have access to the same known future covariates. This is consistent with a *feature-set (information) ceiling* rather than an

architecture ceiling: the eleven domain forecasts largely determine what is predictable, and the purpose-built known-covariate structure of the TFT yields only a small, though consistent, edge (RQ2). Residual diagnostics on the shipped model support this reading; the remaining error is concentrated in a spike tail (the largest-error rows dominate the WAPE numerator) and in per-series heterogeneity (per-series WAPE varies several-fold), neither of which is removed by changing architecture. To isolate the contribution of the known-future covariate directly - rather than inferring it from cross-model convergence - we run a controlled with/without-covariate ablation on the additional dataset in Section 4.5 (Table 3).

4.4 Augmentation Ablation

Since the models in Table 1 converge to nearly the same WAPE regardless of architecture, we tested whether training-time data augmentation could push the TFT past this ceiling. This experiment was applied only to the benchmark dataset. Two techniques were validated at the feature level before training: covariate dropout (randomly withholding a domain covariate per training sample, $p=0.10$) and covariate jitter (Gaussian noise sized at $\sigma=0.20\times$ each covariate’s own standard deviation).

Method	WAPE
TFT (no augmentation)	15.27
TFT + covariate jitter ($\sigma=0.20\times\text{std}$)	15.84
TFT + covariate drop	15.54

Table 2: Augmentation ablation. Evaluation coverage differed across runs; the baseline-matched dropout result provides the strongest evidence.

Covariate dropout, evaluated under the full 3-fold, 5-seed protocol, was consistently worse than its matched baseline across all fold averages and all 5 seeds of the final fold. Covariate jitter showed the same overall trend. A separate single-fold, single-seed baseline is reported only for reference, since it is more sensitive to run-to-run variance. Overall, neither augmentation method improved on the unaugmented TFT. This is consistent with a feature-set ceiling: if performance is mainly limited by the informativeness of the eleven domain covariates, perturbing them during training is unlikely to provide substantial additional gains.

4.5 Additional Dataset

To evaluate whether the findings generalize beyond the benchmark, we apply the same architecture to the Japanese Black Beef Cow Behavior dataset [3]: six cows recorded with tri-axial neck accelerometers at 25 Hz, together with hand-annotated behaviour labels.

Casting the data into the benchmark setup. We treat each contiguous cow recording segment as one series (34 in total). The target is resampled *movement intensity*, defined as the within-second standard deviation of the signal-vector magnitude $\text{SMV} = \sqrt{a_x^2 + a_y^2 + a_z^2}$. The known-future covariate is the behaviour label, interpreted as a planned schedule, together with cyclical calendar features. We forecast 30 seconds from 60–120 seconds of history, using a 6-second stride and holding out the final 30 seconds of each series for validation.

Adaptations. Only dataset-specific preprocessing and temporal scales were changed: accelerometer data were resampled from milliseconds to seconds, the encoder and forecast horizon were rescaled accordingly, and the behaviour label replaced the benchmark’s domain covariates. We used batch size 64 and three seeds (42, 7, 13).

During research and development the decision was made to refer the impact of augmentation on the additional dataset as stated in RQ3 to future research. The proven impact of augmentation on the original dataset implies potential for optimization in that regime.

Method	WAPE	MAE	RMSE
Naive last-value baseline	63.8	0.043	0.091
Series-mean baseline	85.2	0.058	0.093
TFT, without behaviour covariate (ablation)	44.5	0.030	0.059
TFT (with behaviour covariate)	43.0	0.029	0.056

Table 3: Validation results on the additional (cow) dataset, three-seed mean.

Results and discussion. Table 3 reports the transfer results. The TFT reaches WAPE 43.0, beating the naive last-value baseline (63.8) by about 32% and the series-mean baseline (85.2) by half, comfortably clearing the assignment’s minimum bar and confirming that the architecture transfers to a very different, low-data, high-frequency domain. Removing the behaviour covariate (ablation) worsens WAPE to 44.5 and is also consistently worse on MAE and RMSE, so the known-future covariate contributes a real gain. That gain is *modest*, however ($\approx 3\%$ relative WAPE): because the encoder already observes the recent activity signal, target autocorrelation carries most of the predictable structure, and the covariate adds a smaller increment on top. This mirrors the benchmark’s information-ceiling finding (RQ3): once the dominant signal is captured, additional structure yields diminishing returns. We did not enable data augmentation in the reported cow runs; quantifying whether the subsequence-crop augmentation used on the benchmark helps more in this lower-data regime is left as future work.

5 Conclusion

Framing the benchmark around its known-future covariates was the decisive step. A TFT that architecturally separates the known-future pathway gave our best validation result (WAPE 14.48). Answering the research questions explicitly:

- **RQ1 (yes, with a caveat):** the learned models - the TFT, the gated MLP, and the gradient-boosted-tree pipeline - all vastly outperform the naive and seasonal baselines, and the deep TFT attains the best WAPE/MAE; but its margin over the strong tabular baselines is small.
- **RQ2 (largely a ceiling):** the purpose-built TFT beats the generic gated MLP only marginally (14.48 vs. 14.75), and all three learned models share almost identical RMSE - evidence that the fixed known-covariate feature set, not the architecture, sets the performance ceiling.
- **RQ3 (transfers):** on the cow dataset the TFT again beats the naive baseline substantially, and the known-covariate ablation again shows a modest but consistent contribution from the known future covariate - the same information-ceiling pattern.

Limitations. Benchmark error is concentrated in a spike tail and in per-series heterogeneity that a single global model averages over. The transfer dataset is short, high-frequency, and recorded on a single day, so its absolute scores are a proof of concept rather than a tuned system; the cow runs used three seeds and no augmentation, and a benchmark-side feature-removal ablation was not run.

Future work. Per-series or mixture models to address heterogeneity; a distributional loss to model the spike tail inside the network; a controlled data-augmentation study in the low-data cow regime; and a formal benchmark ablation over the known-covariate feature groups.

Contributions

All four members jointly shaped the project: we scoped the problem together, each proposed a candidate additional dataset and prototyped model approaches, and then selected as a team both the additional dataset (Ekinici’s proposal) and the single strongest model to develop further.

- **Zwinge** and **Parashar** did the bulk of the work on the core forecasting model: reframing the task around the known-future covariates and building the LSTM $\rightarrow$ gradient-boosted-tree $\rightarrow$ TFT progression, the cross-validation harness, and the final submission archive.
- **Ekinici** initiated the TFT-model approach and proposed the additional dataset chosen by the group (the cow accelerometer data), supported the further model development, and pursued the evaluation and analysis of the results.
- **Diehl** led the ideation and wrote the exposé, implemented and adapted the additional-dataset (cow) transfer experiment, and - together with Ekinici and Zwinge - polished this report.

AI Tool-Usage

AI tools were used throughout each project phase. Claude Code was used for initial exploratory work with the given data, accelerating the approaches developed by team members. Claude Code and, additionally, OpenAI’s Codex were used to develop the selected TFT approach and adapt it to the additional dataset, combining vibe-coding with directed bug-fixing. Claude Code and Claude Chat were used to cross-check references and polish the written results, exposé, and final report for structure, completeness, and comprehensiveness.

References

- [1] G. Ke et al. “LightGBM: A Highly Efficient Gradient Boosting Decision Tree”. In: *Advances in Neural Information Processing Systems 30 (NIPS)*. 2017. URL: <https://proceedings.neurips.cc/paper/2017/hash/6449f44a102fde848669bdd9eb6b76fa-Abstract.html>.
- [2] B. Lim et al. “Temporal Fusion Transformers for interpretable multi-horizon time series forecasting”. In: *International Journal of Forecasting* 37.4 (2021). DOI: 10.1016/j.ijforecast.2021.03.012.
- [3] H. Ito et al. *Japanese Black Beef Cow Behavior Classification Dataset*. 2022. DOI: 10.5281/zenodo.5849025.
- [4] S. Hochreiter and J. Schmidhuber. “Long Short-Term Memory”. In: *Neural Computation* 9.8 (1997), pp. 1735–1780. DOI: 10.1162/neco.1997.9.8.1735.
- [5] Y. Nie et al. “A Time Series is Worth 64 Words: Long-term Forecasting with Transformers”. In: *Proc. International Conference on Learning Representations (ICLR)*. arXiv:2211.14730. 2023.
- [6] Y. Liu et al. “iTransformer: Inverted Transformers Are Effective for Time Series Forecasting”. In: *Proc. International Conference on Learning Representations (ICLR)*. arXiv:2310.06625. 2024.
- [7] A. F. Ansari et al. “Chronos: Learning the Language of Time Series”. In: *Transactions on Machine Learning Research* (2024). arXiv:2403.07815.
- [8] X. Qiu et al. *A Comprehensive Survey of Deep Learning for Multivariate Time Series Forecasting: A Channel Strategy Perspective*. arXiv:2502.10721. 2025.
- [9] A. Zeng et al. “Are Transformers Effective for Time Series Forecasting?” In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 37. 9. 2023, pp. 11121–11128. DOI: 10.1609/aaai.v37i9.26317.

- [10] Q. Wen et al. “Time Series Data Augmentation for Deep Learning: A Survey”. In: *Proc. 30th International Joint Conference on Artificial Intelligence (IJCAI)*. 2021, pp. 4653–4660. DOI: 10.24963/ijcai.2021/631.